ISEN
ALL IS DIGITAL!
OUEST

TP 4 Data-mining : Forêts aléatoires

Exercice 1 :

On considère les données `spam` (disponibles historiquement dans le package `kernlab` sous R). Nous supposons que les données sont au format CSV.

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier

# Chargement des donnees
spam = pd.read_csv("data/spam.csv")

X = spam.drop('type', axis=1) # type est la colonne contenant 'spam' ou 'nonspam'
y = spam['type']
```

1. En R, la fonction `plot(rf1)` affiche l'erreur OOB (Out-Of-Bag) en fonction du nombre d'arbres. En Python, nous pouvons reconstruire ce comportement en utilisant l'argument `warm_start=True`. Analyser et expliquer le graphe généré par le code suivant :

```python
rf1 = RandomForestClassifier(oob_score=True, warm_start=True, random_state=42)
erreurs_oob = []

for i in range(1, 151):
    rf1.set_params(n_estimators=i)
    rf1.fit(X, y)
    # L'erreur OOB est 1 - le score OOB
    erreurs_oob.append(1 - rf1.oob_score_)

plt.plot(range(1, 151), erreurs_oob)
plt.xlabel("Nombre d'arbres")
plt.ylabel("Erreur OOB")
plt.title("Evolution de l'erreur OOB")
plt.show()
```

2. Construire une nouvelle forêt aléatoire avec le paramètre `max_features=1` (équivalent de `mtry=1` en R) et comparer les erreurs OOB des 2 forêts.

3. Séparer les données en un échantillon d'apprentissage de taille 3000 et un échantillon test de taille 1601 (utiliser `train_test_split` du module `sklearn.model_selection`).

4. Construire deux forêts aléatoires sur les données d'apprentissage : une avec les valeurs par défaut (`max_features="sqrt"`), l'autre avec `max_features=1`.

5. Calculer les erreurs de classification des deux forêts en utilisant les données test (via `accuracy_score` ou en calculant `1 - rf.score(X_test, y_test)`).

6. Utiliser la classe `GridSearchCV` (équivalent de la fonction `train` du package `caret`) pour choisir le paramètre `max_features` dans la grille allant de 1 à 30 par pas de 5 : `range(1, 31, 5)`.

```python
from sklearn.model_selection import GridSearchCV

param_grid = {'max_features': list(range(1, 31, 5))}
grid = GridSearchCV(RandomForestClassifier(random_state=42), param_grid, cv=5)
grid.fit(X_train, y_train)
print("Meilleur parametre :", grid.best_params_)
```

7. Construire un arbre de décision simple (`DecisionTreeClassifier`) sur les données d'apprentissage.
8. Comparer les courbes ROC et les AUC des 2 forêts (par défaut et optimisée) et de l'arbre. (On utilisera l'échantillon test pour calculer les valeurs de probabilités via la méthode `predict_proba` et les fonctions `RocCurveDisplay` ou `roc_curve`, `auc` de `sklearn.metrics`).
9. Calculer les erreurs de classification des 3 méthodes sur l'échantillon de test.
10. Représenter les 10 variables les plus importantes de la meilleure forêt aléatoire à l'aide d'un diagramme en barres.

```python
meilleure_rf = grid.best_estimator_
importances = meilleure_rf.feature_importances_
indices = np.argsort(importances)[::-1]

# Affichage des 10 premieres
plt.bar(range(10), importances[indices][:10])
plt.xticks(range(10), X.columns[indices][:10], rotation=45)
plt.title("10 variables les plus importantes")
plt.show()
```

11. En R, on utilise le package `ranger` pour avoir une implémentation plus rapide que `randomForest`. En Python, comparer les temps de calcul de `RandomForestClassifier` avec 1 seul cœur (`n_jobs=None`) et avec tous les cœurs disponibles (`n_jobs=-1`). Vous pouvez utiliser le module `time` de Python pour mesurer la durée d'entraînement.

Exercice 2 :

On considère les données *internet advertisements* disponibles sur le dépôt UCI (`https://archive.ics.uci.edu/ml`

1. Importer les données. Étant donné la nature de la base, vous devrez probablement forcer certains types et traiter les données manquantes représentées par `"?"`. Expliquer le problème (classification de bannières publicitaires).
2. Nommer `Y` la variable à expliquer (généralement la dernière colonne) et supprimer les observations qui contiennent des données manquantes (avec `replace('?', np.nan).dropna()` par exemple).
3. Séparer les données en un échantillon d'apprentissage de taille 2000 et un échantillon test de taille 359 (soit un total de 2359 observations après nettoyage).
4. Calculer la variance des variables explicatives de l'échantillon d'apprentissage : `X_train.var()`. Interpréter (certaines variables ont-elles des échelles très différentes ?).
5. Construire un arbre (`DecisionTreeClassifier`), une Analyse Discriminante Linéaire (`LinearDiscriminant` du module `sklearn.discriminant_analysis`) et une forêt aléatoire (`RandomForestClassifier`) sur les données d'apprentissage.
6. Comparer les 3 méthodes avec la courbe ROC puis l'AUC sur l'échantillon test.

7. Faire le même travail (comparaison des performances globales) en utilisant une validation croisée à 10 blocs (`cross_val_score` avec `cv=10` sur l'ensemble des données nettoyées).